

Reinforcement Learning

Introduction

Textbook Chapter 1

Michael Hahsler, SMU

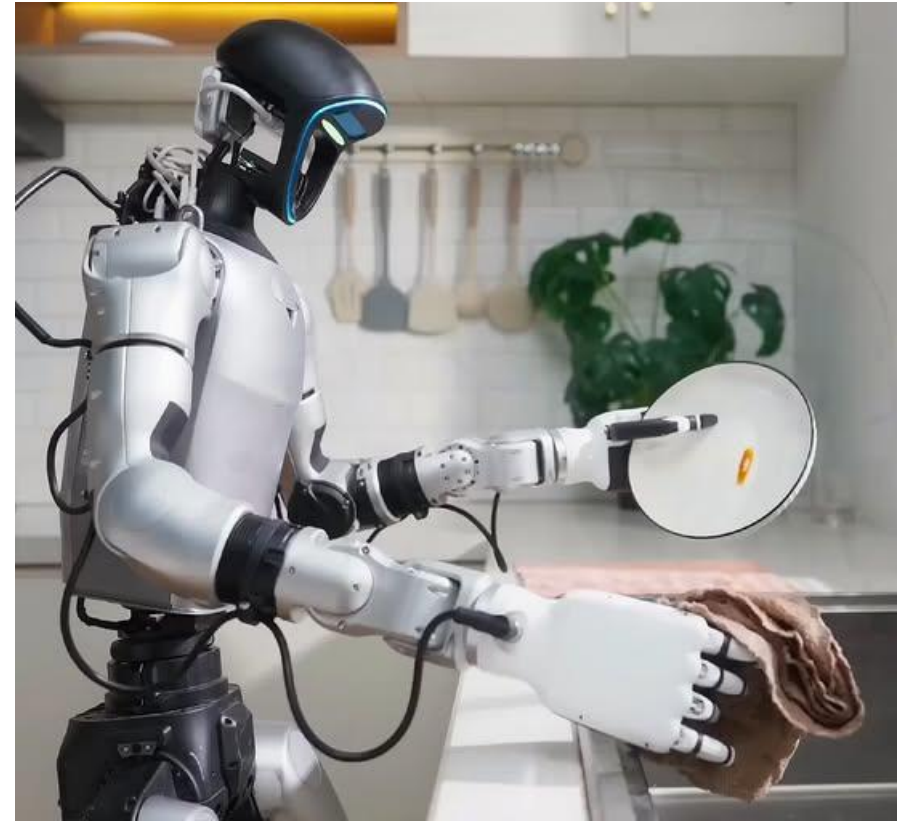
Based on slides by Oliver Wallscheid (Paderborn University) and material by David Silver (University College London)



How Do We Build Intelligent Systems?

Options:

- Explicitly program behavior: **Rule-based agent, planning**
- Learn behavior from labeled data offline: **Supervised learning**
- Learn behavior using feedback from interacting with the environment (online): **Reinforcement Learning**



Topics of this Course

- **Introduction to reinforcement learning**
- **Markov decision processes**
- **Part I: Tabular Methods**
 - Dynamic programming
 - Monte Carlo methods
 - Temporal-difference learning
 - Multi-step bootstrapping
 - Planning and learning with tabular methods
- **Part II: Approximate Solution Methods**
 - Prediction and Control using Approximation
 - Eligibility Traces
 - Policy Gradient Methods
- **Part III: Modern RL Methods**
 - Deep Reinforcement Learning
 - Current Applications

Recommended Readings

- Richard S. Sutton, Andrew G. Barto,
[*Reinforcement Learning: An Introduction*](#),
2nd edition, MIT Press, Cambridge, MA, 2018.
- Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare and Joelle Pineau,
[*An Introduction to Deep Reinforcement Learning*](#),
Foundations and Trends in Machine Learning, 11:3-4, pp 219-354,
2018.

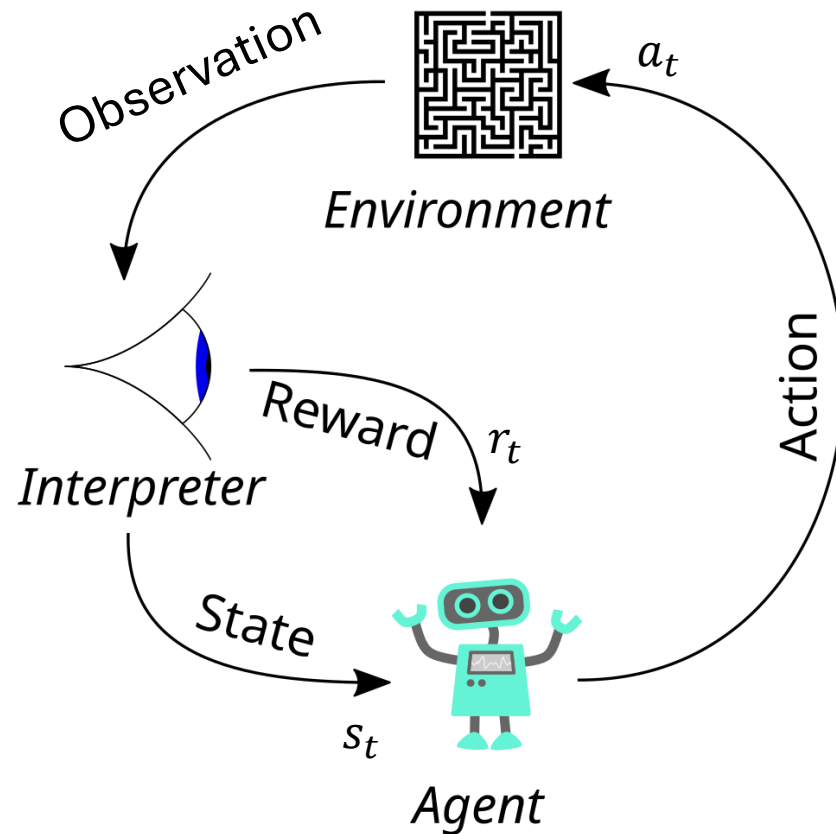
Table of Contents

- What is RL?
- Elements of RL
 - Reward
 - States
 - Actions and policies
- A small example
- RL and planning

A hand is holding a yellow, bone-shaped treat in the upper left corner. A white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe. The background is a solid light gray.

What is RL?

The Basic Reinforcement Learning Structure



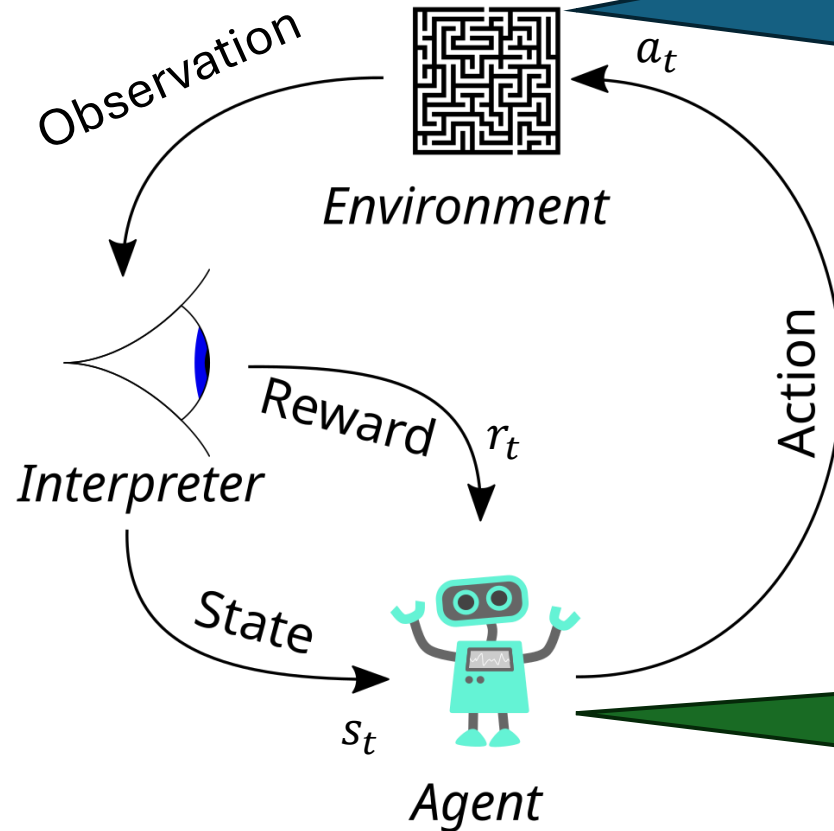
The basic RL operation principle
(derivative of www.wikipedia.org, CC0 1.0)

Goal: Map a situation to an action to maximize the reward

Key characteristics:

- **Uncertainty about the environment** leads to a need for sensing.
- **No supervisor:** discover actions by trying them (data-driven)
- **Sequential decision making:** Agent actions affect subsequent data. Agent maximizes over future (delayed) rewards which need to be estimated!
- Action discovery and maximization lead to a tradeoff: **exploit vs. explore**

Agent and Environment



We often model the interpreter as part of the environment.

At each step t the environment:

- Receives an action a_t .
- Emits an observation s_{t+1} .
- Emits a reward r_{t+1} .

The time increments $t \leftarrow t + 1$.

At each step t the agent:

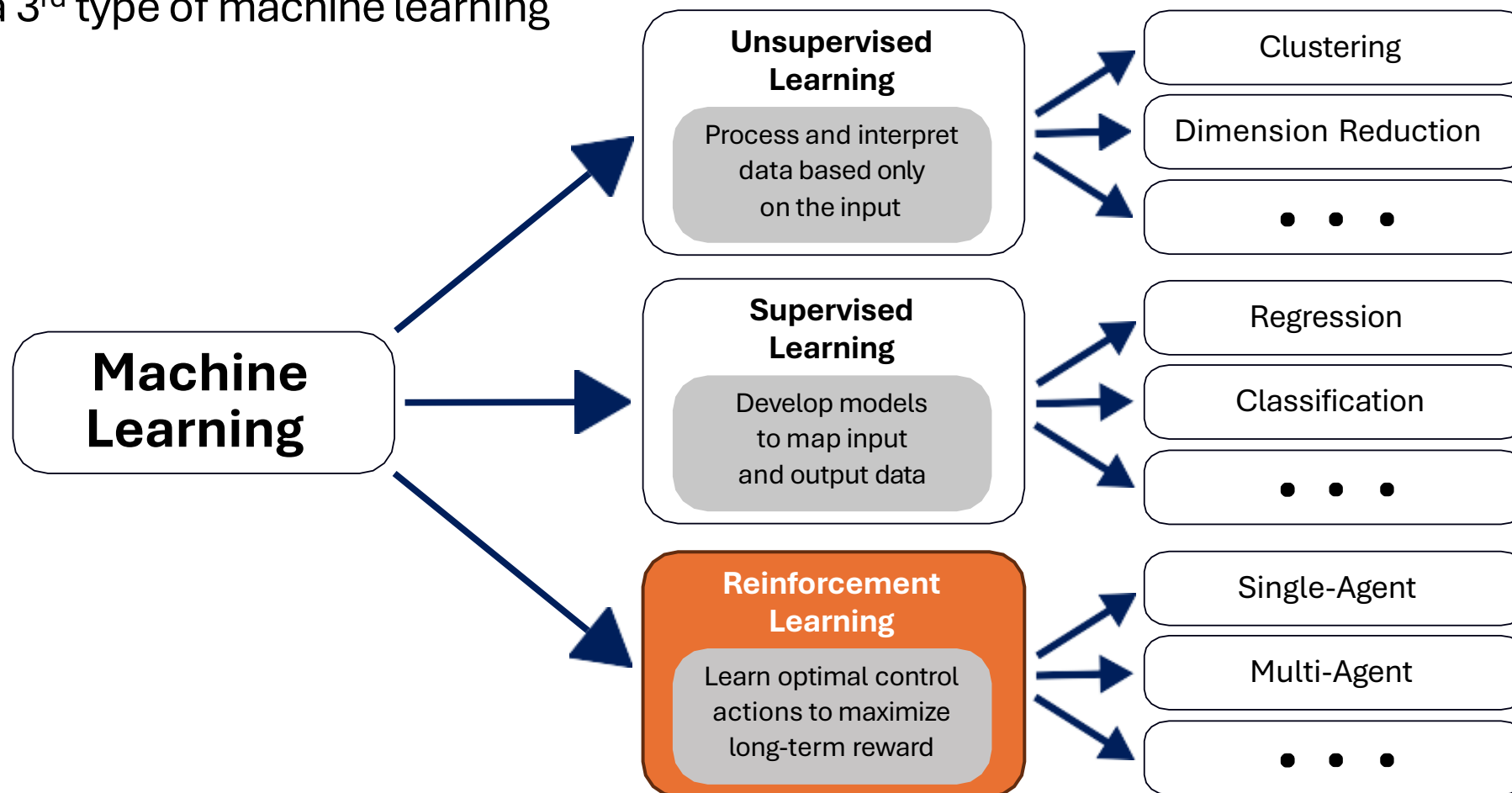
- Receives the state s_t .
- Receives a reward r_t .
- Picks an action a_t .

Remark on time: In AI, we often assume a time delay of one unit between executing the action and receiving the next state as well as the reward. This is called a discrete-time process (vs. a continuous-time process).

RL and Machine Learning

Machine learning is studied in AI to learn from examples.

RL is a 3rd type of machine learning



Many Faces of Reinforcement Learning

Optimal sequential decision making

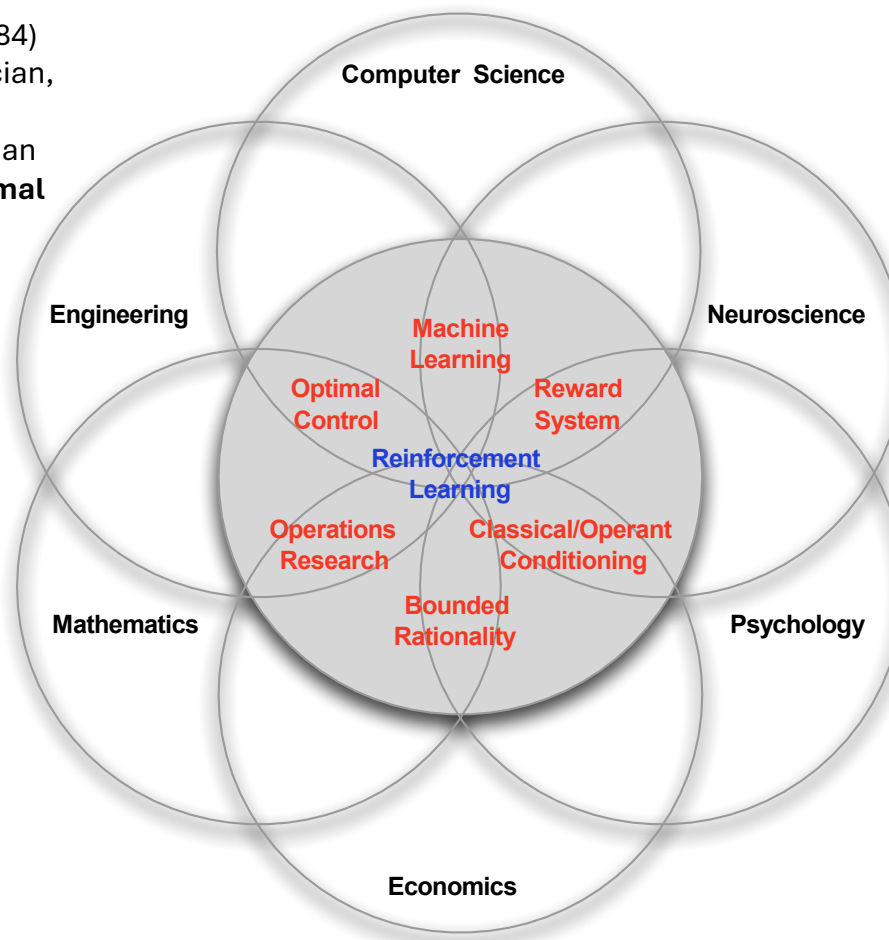


Richard E. Bellman (1920-1984)
American applied mathematician, who introduced **dynamic programming** and the Bellman equation. Foundation of **optimal control theory**.

Stochastic process formalism



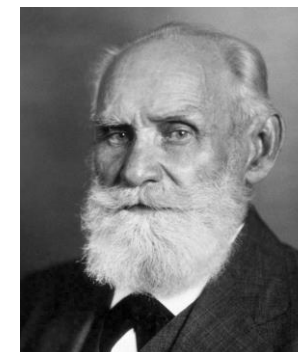
Andrei Markov (1856-1922)
Russian mathematician developed the groundwork for **Markov chains** to model **dynamical systems**.



RL and its neighboring domains

(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/))

Classical conditioning



Ivan Pavlov (1849-1936)
Russian physiologist and Nobel laureate who discovered **classical conditioning** through his experiments with dogs.

Contemporary application examples

RL has led to superhuman performance for many games and simple toy examples

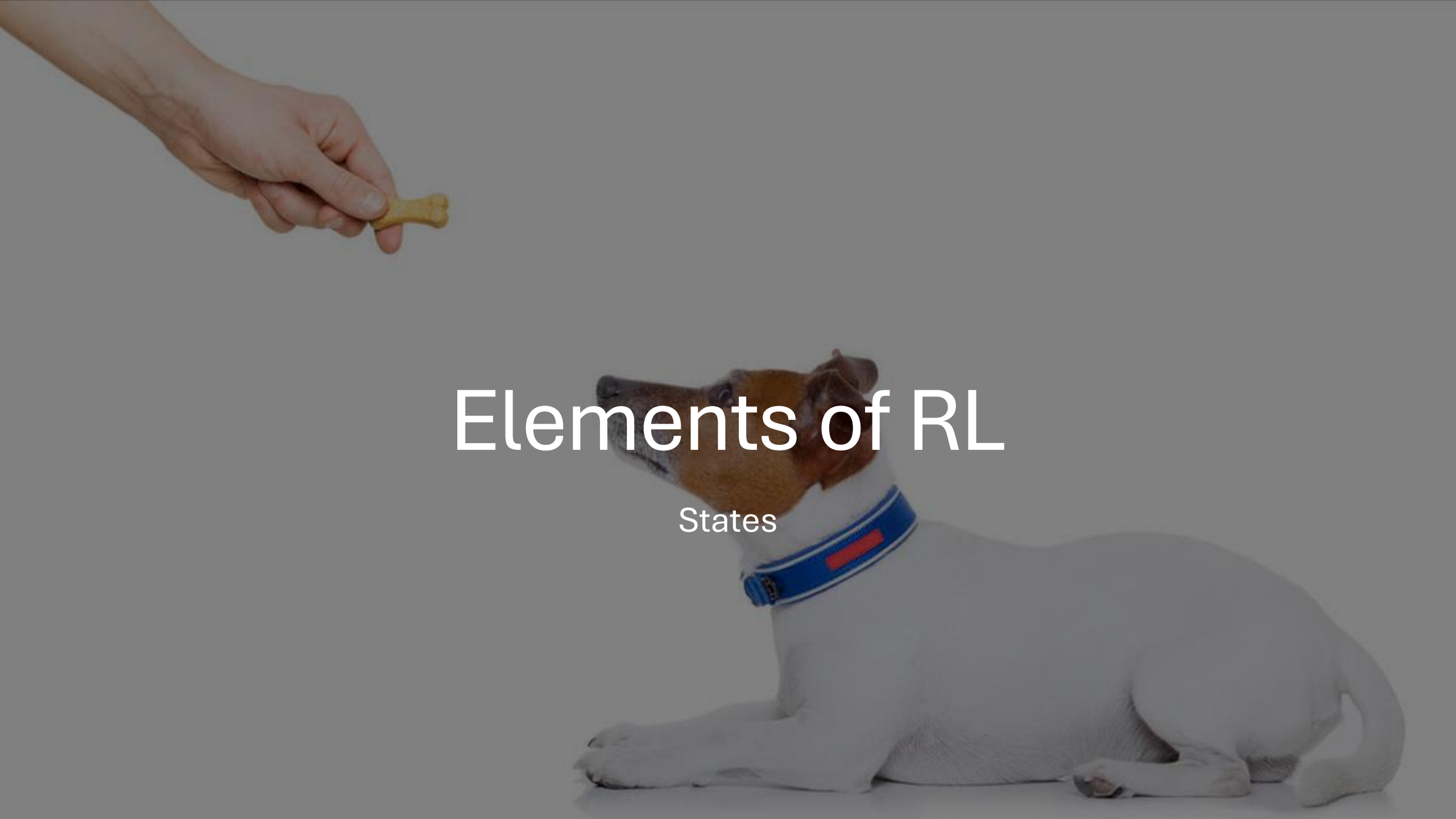
- [Playing Atari Breakout](#)
- [Play strategy board game Go at super-human performance](#)
- [Simulated classic control problems](#)
- [Swinging-up and balance a cart-pole / an inverted pendulum](#)

Using RL for real applications is promising, but much more difficult. Here are some examples:

- [Controlling electric drive systems](#)
- [Flipping pancakes with a roboter arm](#)
- [Drifting with a RC-car](#)
- [Driving an autonomous car](#)
- [Nuclear fusion reactor plasma control](#)
- [Training chat bots \(like chatGPT\)](#)
- ...

Reasons for difficulty:

- Real environments are complicated.
- Sensors (observations) are noisy.
- We often have partial observability.
- The number of different actions can be large.
- Measuring reward can be difficult.
- No access to a fast, perfect simulation environment.

A hand holding a bone over a dog. A person's hand is visible in the upper left corner, holding a small, yellow, bone-shaped treat. Below the hand, a white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe. The background is a solid light gray.

Elements of RL

States

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

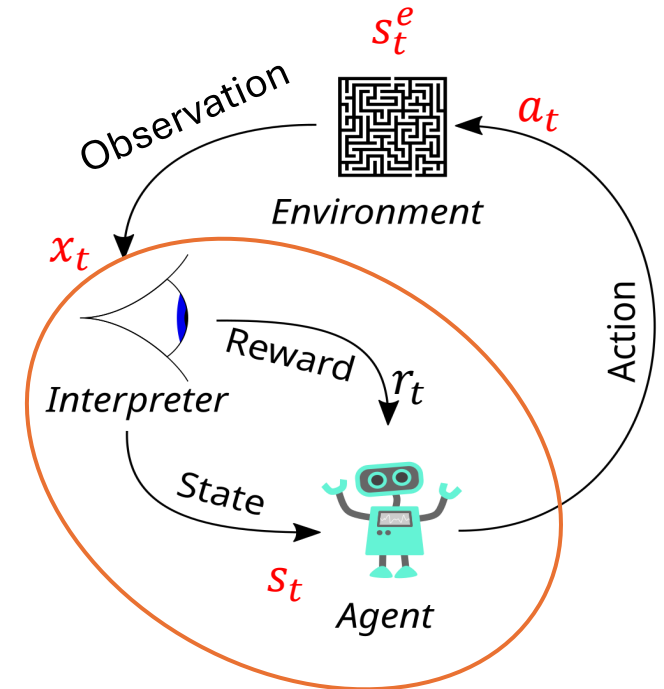
G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*
$\bar{V}_t(s)$	expected approximate action value
U_t	target for estimate at time t

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a
	expected immediate reward from state s after action a
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

State

- The environment has an **environment state** s_t^e that contains all information needed to determine how it will react to actions:
 - Physical states, e.g., current car velocity and direction
 - Game states, e.g., current chess board
 - Financial states, e.g., stock market prices, sentiment,...
- We call such a state an **information state**.
- **Agent state** s_t is the agent's internal representation of the situation.
 - What it knows about the environment state?
 - It is the basis of choosing the next action.
 - Can be incomplete and noisy based on observations x_t
 - Can include the agent's internal memory (e.g., what is my current strategy), including beliefs, learned features, etc.
 - Can be compressed (often using an artificial neural network).
- With state, we will mean the agent state and we will use the random variable S_t .
- The state typically has a factored representation with discrete and continuous variables.



The agent design often includes designing the interpreter

Degree of Observability

Full observability

- Agent directly observes the full environment state (e.g., $s_t = s_t^e$).
- s_t is an information state: Markov Decision Process (MDP)

Partial observability

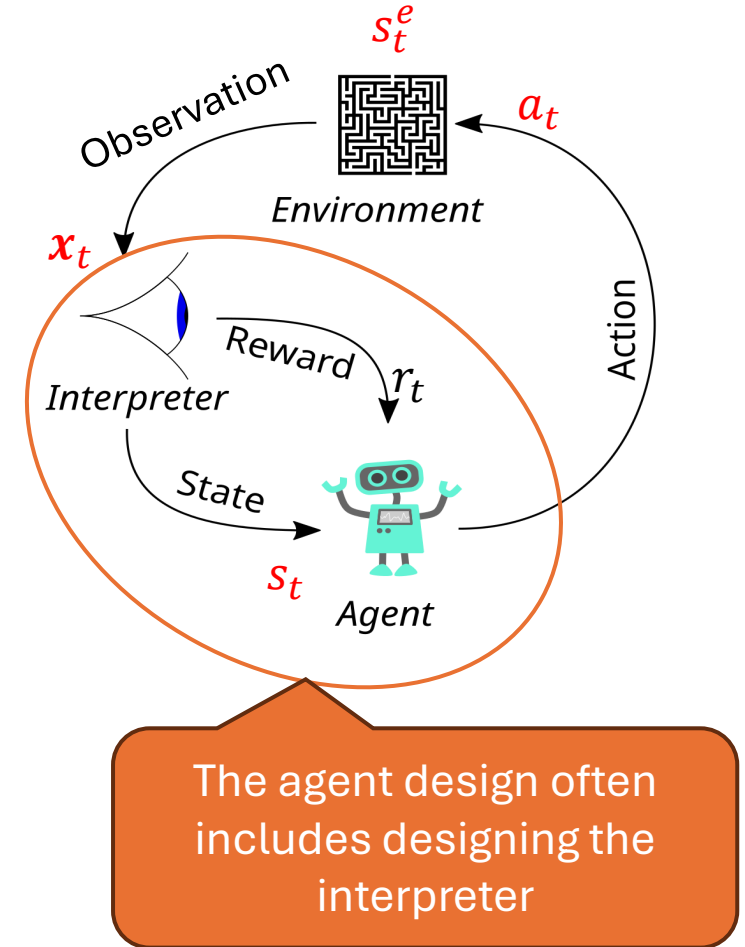
- Agent does not have full access to the environment state but observes state features $x_t = f(s_t^e)$.
- Agent may try to reconstruct the complete state information (state estimation)

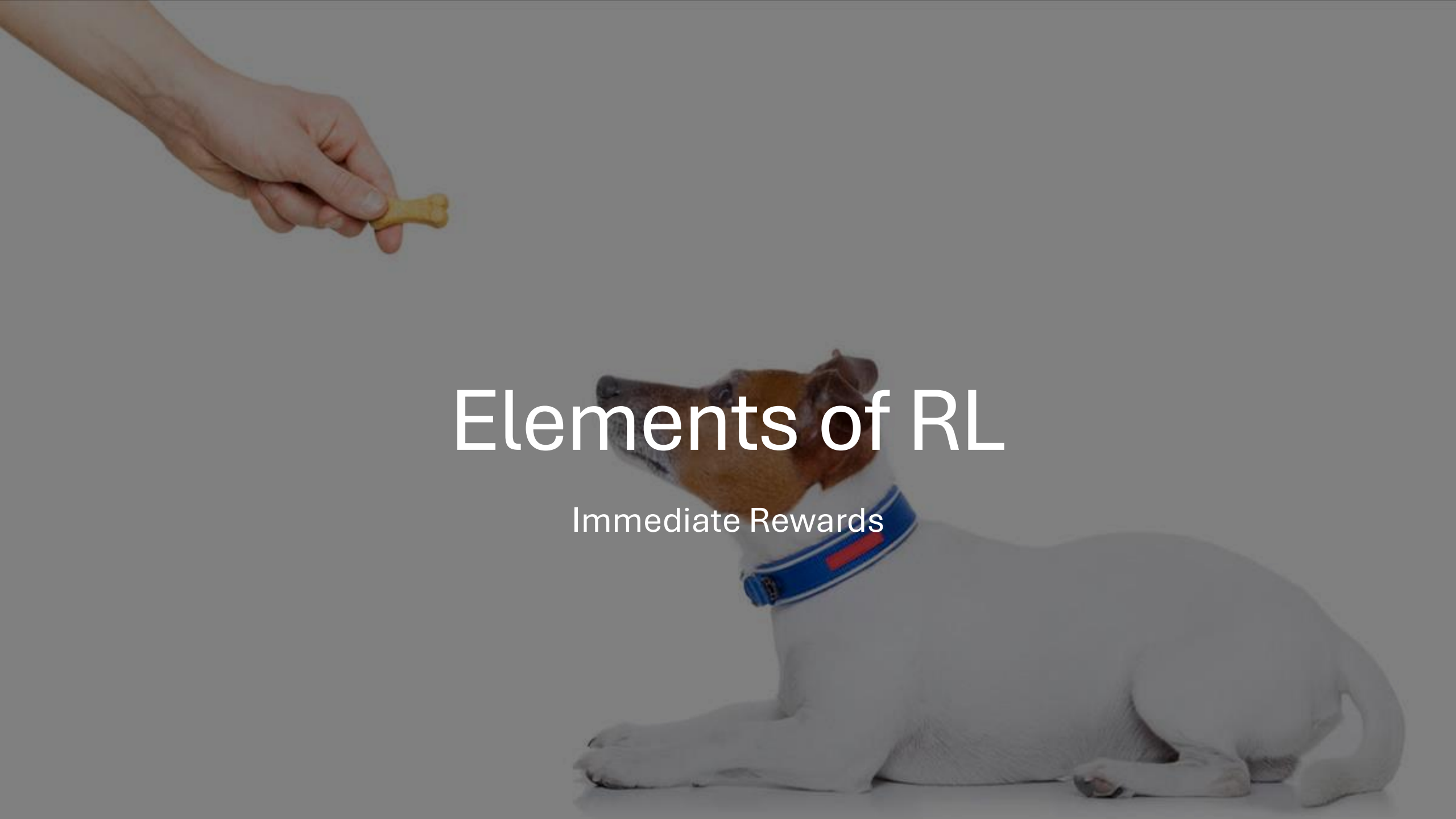
$$s_t = \widehat{s}_t^e = \text{update}(x_t, a_t, \widehat{s}_{t-1}^e)$$

- This leads to Partially Observable Markov Decision Processes (POMDP)

POMDP examples

- Self-driving cars with only cameras and no radar/lidar (to reduce cost)
- Poker game (opponents' cards are unknown)
- Human health status (we cannot sense everything and the detailed system dynamics are unknown)



A hand holding a bone-shaped treat over a dog. The background is a solid light gray. A hand from the top left corner holds a yellow, bone-shaped treat. Below it, a white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe.

Elements of RL

Immediate Rewards

Reward

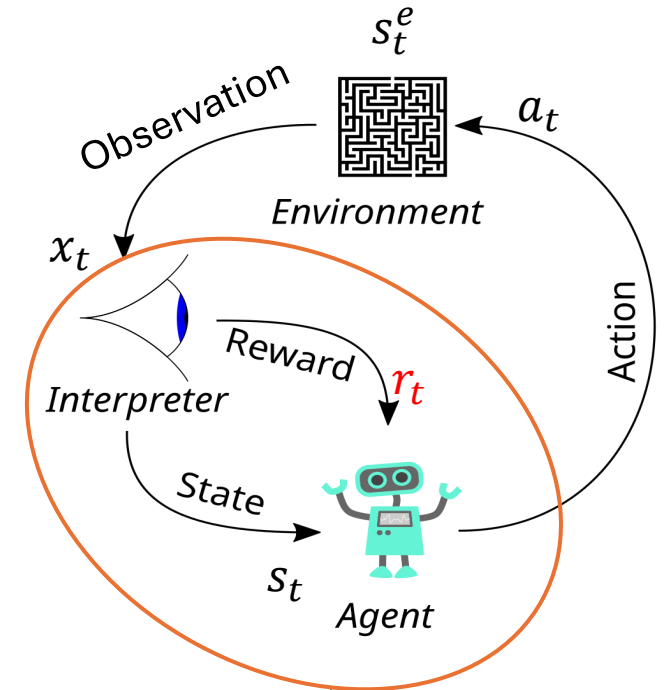
- An **immediate reward** is a scalar random variable R_t with realizations $r_t \in \mathbb{R}$ that the agent “receives” at time t .
- **Reward function:** The reward is often defined as a function of the current state s_t . Examples:
 - 1 point for winning a game (state is a winning board).
- The **agent’s task** is to choose actions to maximize the long-term reward called the return.

Reward hypothesis: all goals and purposes, whether in AI or humans, can be reduced to maximizing the expected value of a cumulative scalar reward signal:

$$\max \mathbb{E} \left[\sum_{t=0}^{\infty} w_t R_t \right]$$

That is, the **reward structure fully determines the goal**.

- The **designer’s task** is to design the reward signal based on the goal.
- **Reward shaping:** The reward signal can be manipulated to improve learning. E.g., add a reward for intermediate goals, such as achieving 3 in a row in Connect-4.



The agent design often includes designing the interpreter

Reward Examples

Flipping a pancake:

- Pos. reward: catching the flipped pancake
- Neg. reward: dropping the pancake on the floor

Stock trading:

- Trading portfolio monetary value

Playing Atari games:

- Highscore value at the end of a game episode

Driving an autonomous car:

- Pos. reward: getting safely from A to B without crashing
- Neg. reward: hitting another car, pedestrian, bicycle,...

Classical control task (e.g., electric drive, inverted pendulum,...):

- Pos. reward: following a given reference trajectory precisely
- Neg. reward: violating a system constraint or producing a large control error

Reward Characteristics

Rewards are highly dependent on the given problem:

- Actions may have **short and long-term consequences**.
 - The reward for a certain action may be delayed, creating the credit assignment problem.
 - Examples: Stock trading, strategic board games,...
- Rewards can be **positive and negative values**.
 - Certain situations (e.g., car hits a wall) might lead to a negative reward (= a cost).
- Exogenous impacts might introduce **stochastic reward components**.
 - Example: A wind gust pushes an autonomous helicopter into a tree.

Issues With the Reward Function

“Be careful what you wish for - you might get it” (proverb)

“...it grants what you ask for, not what you should have asked for or what you intend.”
(Norbert Wiener, American mathematician)



Midas and daughter (good as gold)

(source: www.flickr.com, by [Robin Hutton](#) CC BY-NC-ND 2.0)

Designing an effective reward function is crucial for successful RL applications!

It is hard to define a good reward function:

- The RL agent's learning process heavily depends on the reward distribution over time.
- Alignment problem and reward hacking.

Return Definitions

Definition: Return is the sum of rewards that the agent seeks to maximize.

Episodic tasks

- A problem which naturally breaks into subsequences (episodes) with a finite horizon.
- Examples: most games like chess, solving a maze,...
- The return becomes a finite sum with horizon T :

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T = \sum_{k=1}^T R_{t+k}$$

Continuing tasks

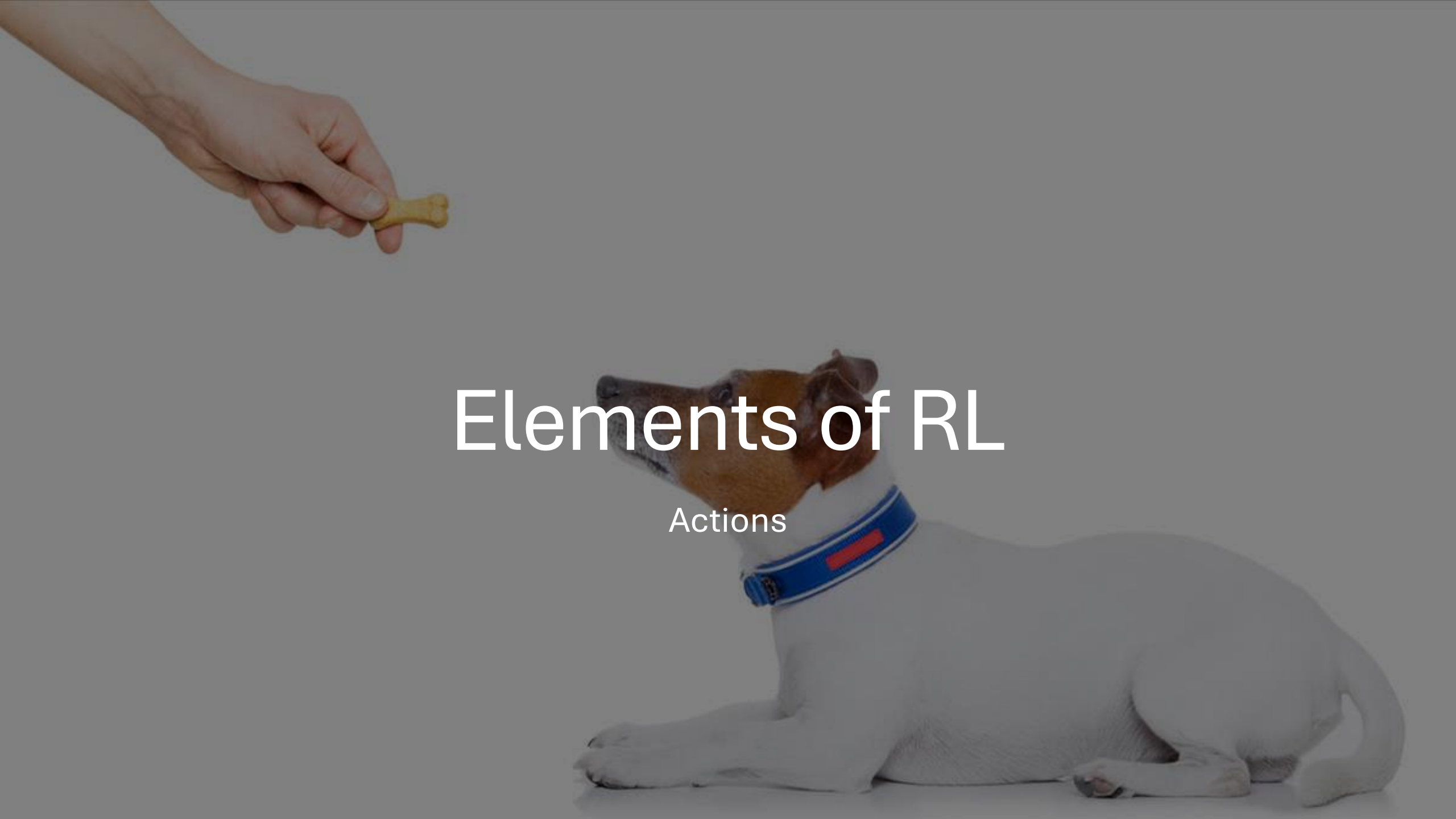
- A problem without a natural end (infinite horizon).
- Example: A smart thermostat controlling room temperature
- We use **discounting** and have a **recursive relationship**:

$$\begin{aligned} G_t &= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \\ &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

- The discount rate $0 \leq \gamma \leq 1$ is a weight prevents infinite sums.

Strategic viewpoint of discounting:

- If $\gamma \approx 1$: agent is farsighted.
- If $\gamma \approx 0$: agent is shortsighted (only interested in immediate reward).

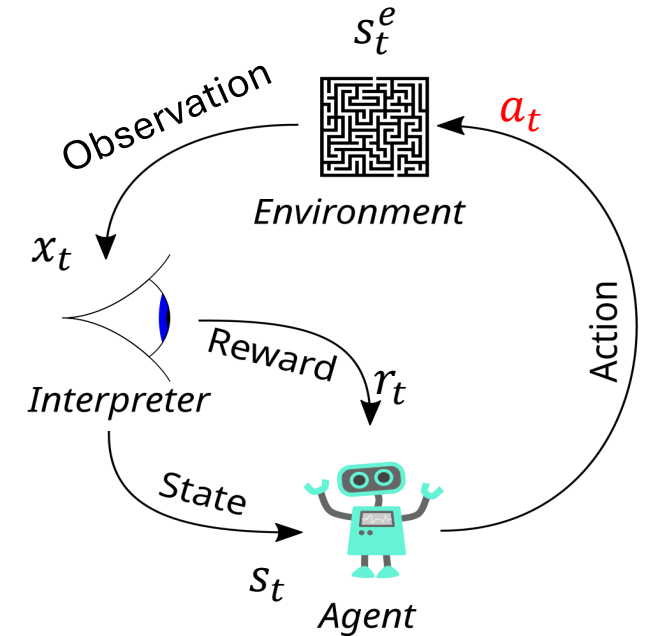
A hand holding a bone over a dog. A person's hand is visible in the upper left corner, holding a yellow, bone-shaped treat. Below the hand, a white dog with brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe. The background is a solid light gray.

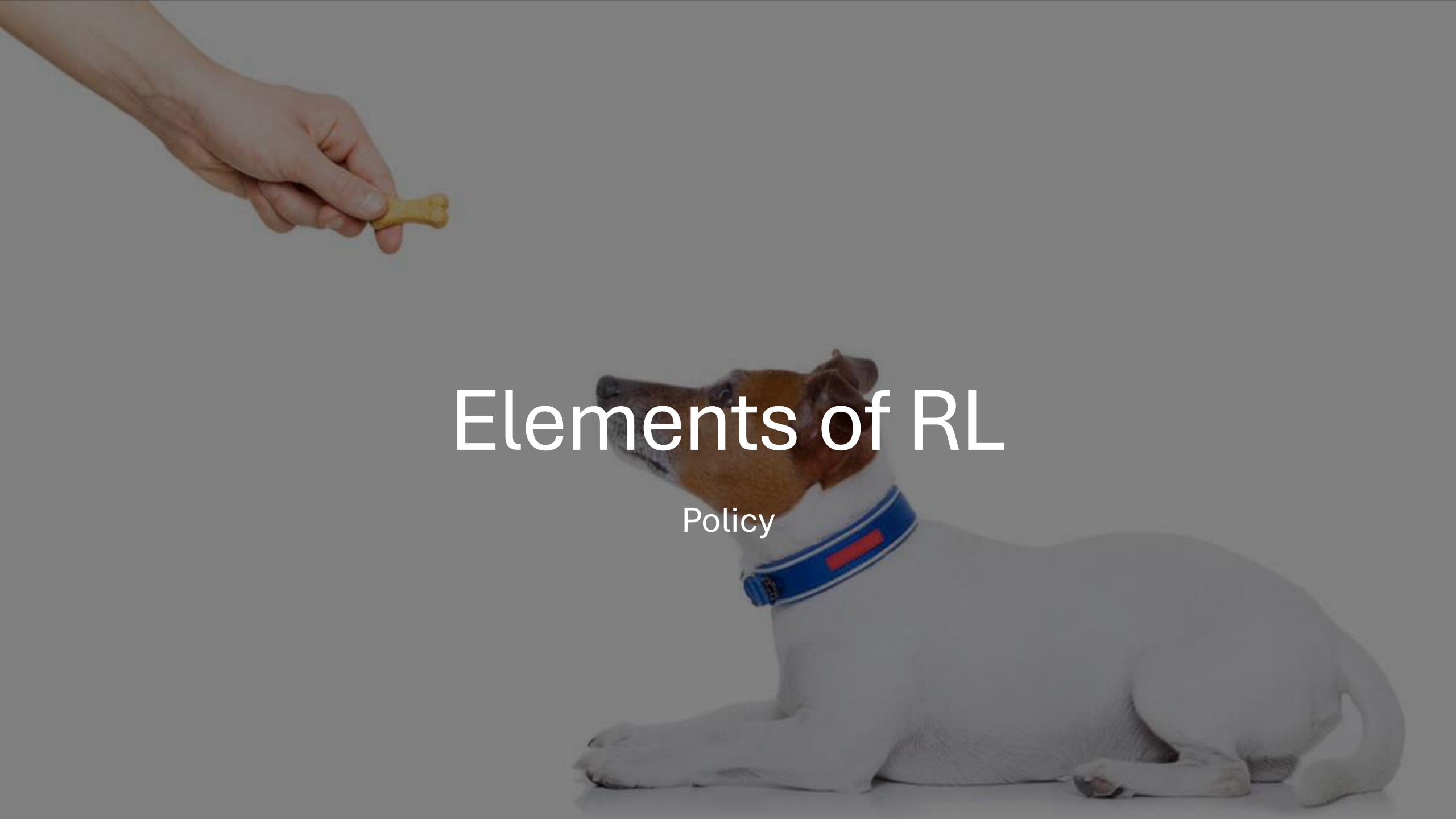
Elements of RL

Actions

Action

- Choosing actions is the agent's mean to maximize its long-term reward.
- We use the random variable A_t or a known action $a_t \in \mathcal{A}$
- **Finite action set (FAS):** $\mathcal{A}$ is a finite set. Often available actions depend on the current state $a_t \in \mathcal{A}(s_t)$.
- **Continuous action set (CAS):** $\mathcal{A} = \mathbb{R}^m$
- Examples:
 - Take a card during a Blackjack game (FAS)
 - Steering while driving an autonomous car (CAS)
 - Buy stock options for your trading portfolio (FAS/CAS)



A hand holding a bone over a dog. A hand is visible in the upper left corner, holding a small, yellow, bone-shaped object. Below the hand, a dog is lying down, looking up at the bone. The dog has white fur with brown patches on its head and ears. It is wearing a blue collar with a red stripe. The background is a solid light gray.

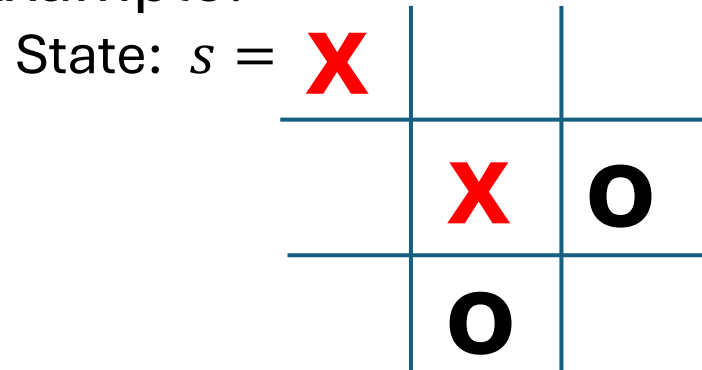
Elements of RL

Policy

Deterministic Policy

- A policy is the rule of how an agent chooses actions.
- The choice depends on the current knowledge of the agent, which is the current state s .
- A deterministic policy prescribes an action for each state:
$$\pi: \mathcal{S} \rightarrow \mathcal{A} \quad \text{or} \quad a = \pi(s)$$

- Example:



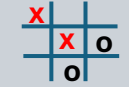
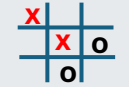
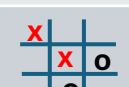
Action a : x always plays bottom right corner.

Stochastic Policy

- A mapping from states to the probability distribution of selecting each action:
 $\pi: \mathcal{S} \rightarrow \Delta(\mathcal{A})$

often written as $\pi(s|a)$ giving the probability of choosing a in state s .

- Example:** Probability distribution indicating to play bottom right most of the time.

State s	Action a	Probability $\pi(s a)$
	Top center	.1
	Top right	.1
	Middle left	.1
	Bottom left	.1
	Bottom right	.6

Note: The deterministic policy is just a special case with 1 for the chosen action and 0 for all others.

Policy: Exploration and Exploitation

- The agent needs to learn the optimal policy using the reward signal.
- At any point in time, the agent can choose actions to perform:
 - **Exploitation:** maximize return using current information about the environment (= use current best policy).
 - **Exploration:** try out a new action to find out more about the environment. Has the potential to improve the policy.
- **Trade-off problem:** What is the best split between both strategies?

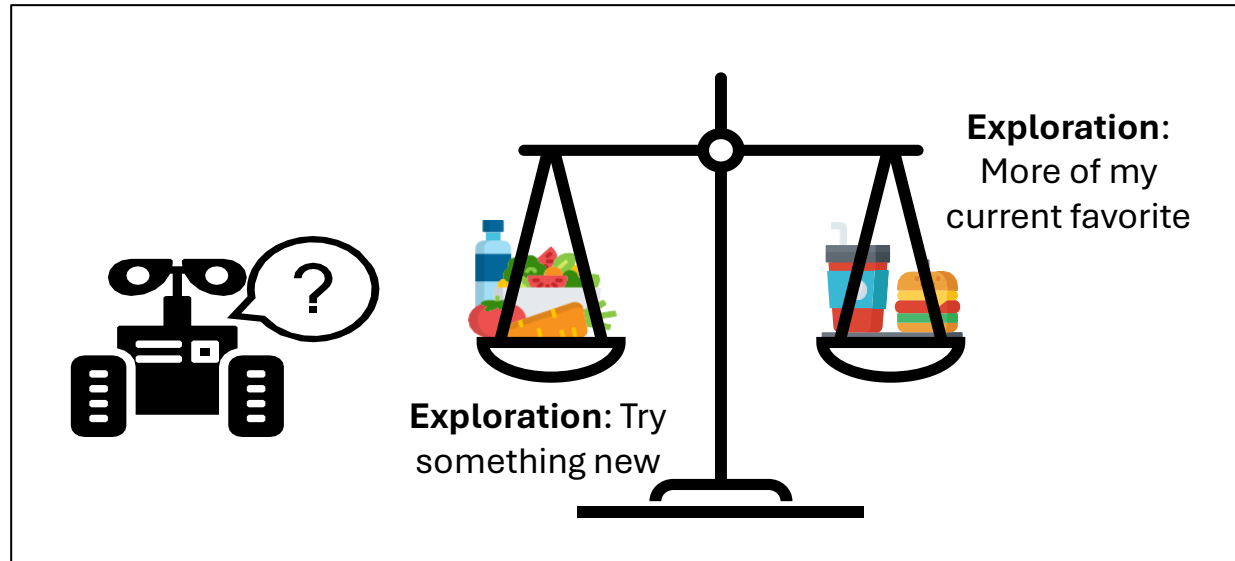
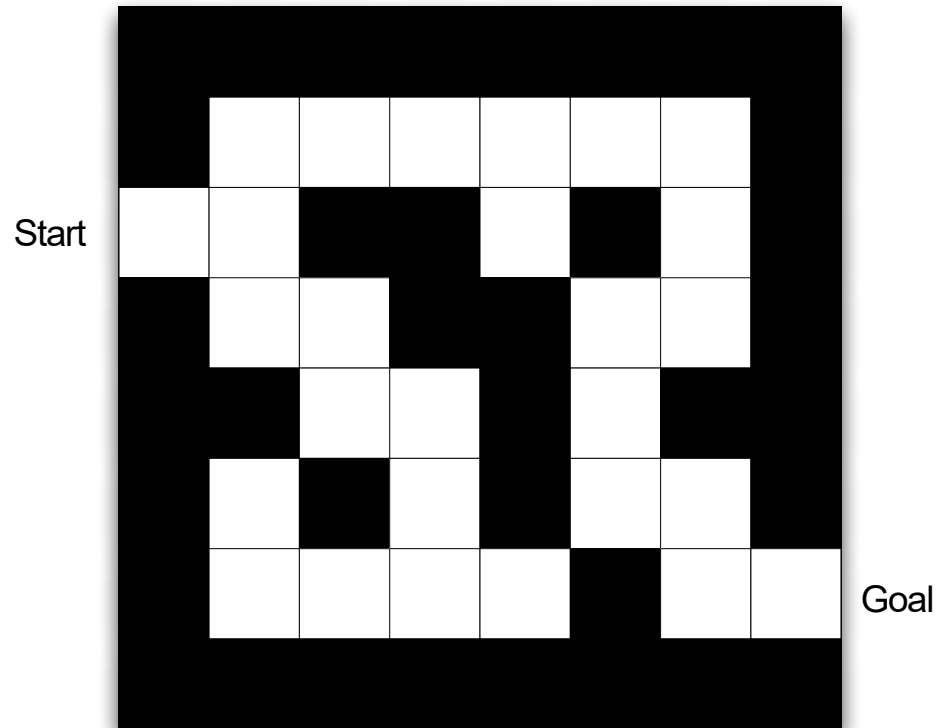


Figure: The exploration-exploitation dilemma

A Small Example

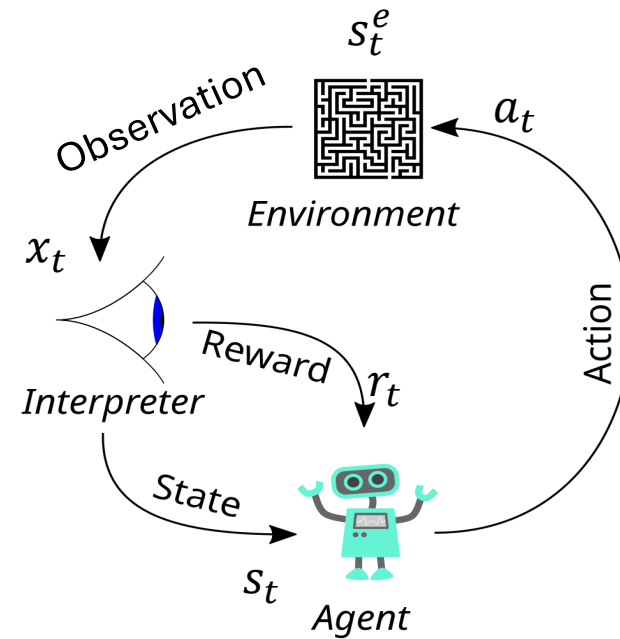


Maze Example



Maze setup

(source: D. Silver, “Reinforcement learning”, 2015. [CC BY-NC 4.0](#))

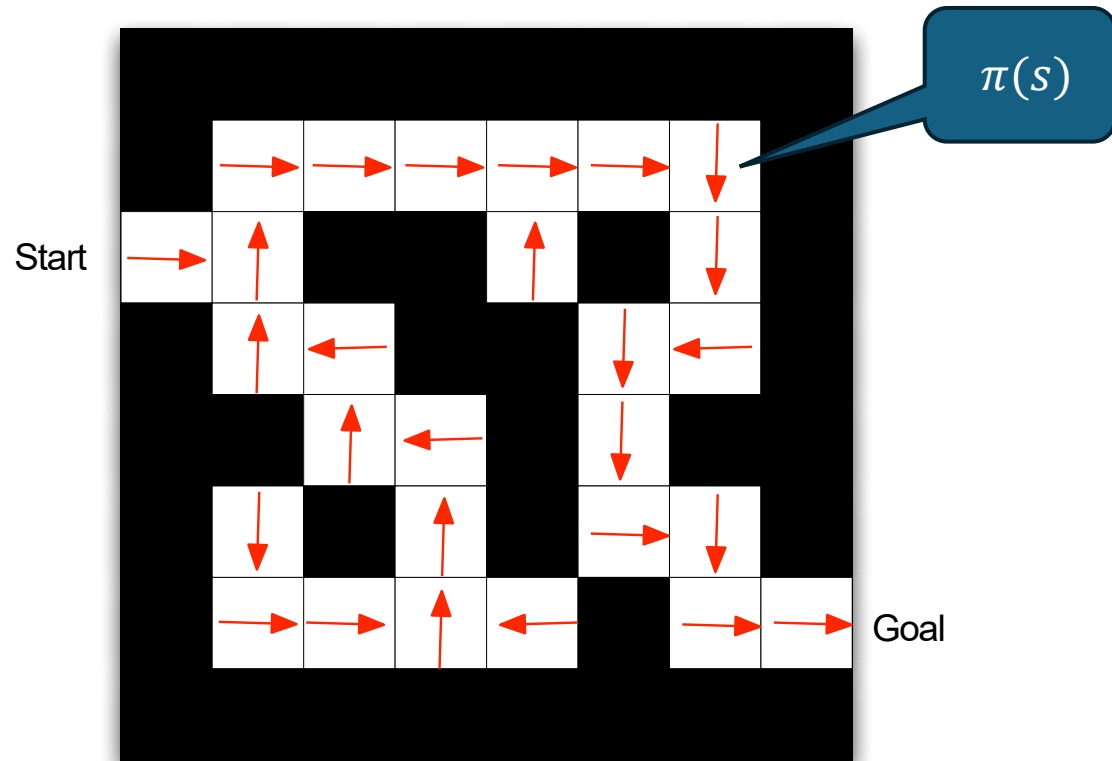


Problem statement

- State: agent's location is observable
- Actions: $a_t \in \{N, E, S, W\}$
- Transition model: deterministic
- Immediate Reward: $r_t = -1$
- At goal: episode terminates

Since the agent wants to maximize the total reward, it needs to minimize the number of steps to get to the goal!

Maze Example: RL-Solution as a Policy



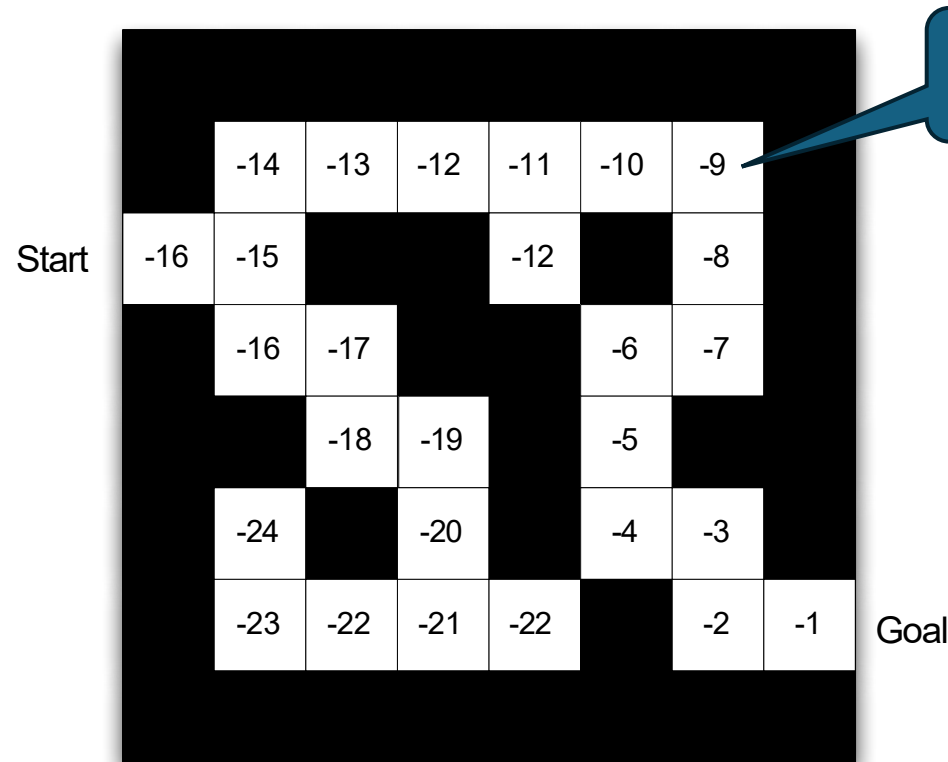
Key characteristics of the policy:

- Specify an action (direction) for each state (position in the maze).
- Policy can be explicitly stored as a table.

Maze setup: Arrows represent policy $\pi(s)$

(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](#))

Maze Example: RL-Solution as a Value Function



$v_{\pi}(s)$

The value function estimates the expected long-term reward in a state under a policy.

$v_{\pi}(s)$

Key characteristics

- The agent chooses actions to move to states with better state value.
- The policy is implicitly defined by the state values.

Numbers represent value $v_{\pi}(s)$: Each step costs 1, i.e. $r = -1$

(source: D. Silver, "Reinforcement learning", 2015. [CC BY-NC 4.0](#))

A hand holding a bone over a dog. A person's hand is visible in the upper left corner, holding a small, yellow, bone-shaped treat. Below the hand, a dog with white fur and brown patches on its head and ears is lying down, looking up at the treat. The dog is wearing a blue collar with a red stripe. The background is a solid light gray.

RL and Planning

Model-based vs. Model-free Methods

Model-based vs. Model-free RL

Model-based RL

- Known Environment: The agent has a complete model of the environment.
 - Transition model
 - Reward model
- The agent does not need to interact with the environment.
- The agent can use the model to **plan** offline and find the optimal policy.
- Issues:
 - Computational complexity
 - Model errors compound

Model-free RL

- Unknown environment: Model is unknown.
- The agent needs to interact with the environment.
- Agent uses online **trial & error** by trying actions and updating its policy based on the received reward.
- Issues:
 - Sample efficiency
 - Exploitation vs. exploration

Side Note: Model Predictive Control (MPC)

MPC are methods for optimal control of a dynamical system. They are

- **model-based**,
- **continuous**,
- solved **offline** (often with a closed-form solution), and
- results in **stable** and robust solutions.

- Example: **Linear Quadratic Regulator (LQR)**
- Minimize the infinite-horizon quadratic continuous-time cost function

$$J = \frac{1}{2} \int_0^{\infty} [\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t)] dt$$

- $\mathbf{x}$ is the state (vector)
- $\mathbf{u}$ is the action.
- $\mathbf{Q}$ and $\mathbf{R}$ are the reward (cost) model

- With linear time-invariant first-order dynamic constraints

$$\mathbf{x}(t+1) = \mathbf{A} \mathbf{x}(t) + \mathbf{B} \mathbf{u}(t)$$

- $\mathbf{A}$ and $\mathbf{B}$ are the transition model.

- This can be solved to create a feedback control law

$$\mathbf{u}(t) = \mathbf{K} \mathbf{x}(t)$$

- MPC is very popular in robotics (mechanical/electrical engineering).

This course will focus on RL



Summary: What You Should Know

- Understand the basic **RL interaction loop**: agent, environment, interpreter.
- **Basic RL vocabulary**: state, action, immediate reward, return, policy, value function.
- Appreciate the significance of proper **reward formulation**.
- Differentiate between **model-based** (planning) and **model-free** reinforcement learning methods.